IPFS 설정 프로파일 시스템 문서화

개요

IPFS 설정 프로파일 시스템은 다양한 사용 사례에 맞게 노드를 최적화할 수 있는 사전 정의된 설정 템플릿을 제공합니다. 각 프로파일은 특정 환경이나 요구사항에 맞춰 설정을 자동으로 조정하여 사용자가 복잡한 설정을 수동으로 변경할 필요를 줄여줍니다.

프로파일 시스템 아키텍처

핵심 구조체

```
// Profile은 설정 변환기와 프로파일 설명을 포함합니다
type Profile struct {
Description  string  // 프로파일 기능에 대한 간략한 설명
Transform Transformer // 설정에 프로파일을 적용하는 함수
InitOnly  bool  // 초기화 시에만 적용 가능한지 여부
}

// Transformer는 설정을 받아 필터를 적용하는 함수입니다
type Transformer func(c *Config) error
```

프로파일 적용 메커니즘

```go
func applyProfiles(conf *config.Config, profiles string) error {
if profiles == "" {
return nil
}

// 쉼표로 구분된 프로파일들을 순차적으로 적용
for _, profile := range strings.Split(profiles, ",") {
transformer, ok := config.Profiles[profile]
if !ok {
return fmt.Errorf("invalid configuration profile: %s", profile)
}

if err := transformer.Transform(conf); err != nil {
return err
}
}
return nil
}
```

네트워킹 프로파일

server 프로파일

목적: 공용 IPv4 주소를 가진 서버 환경에서 실행할 때 권장되는 설정

주요 변경사항:

- 로컬 호스트 발견 비활성화
- 사설 IP 주소 필터링 강화
- NAT 포트 매핑 비활성화

```
"server": {
Description: `Disables local host discovery, recommended when
running IPFS on machines with public IPv4 addresses.`,
Transform: func(c *Config) error {
// 사설 IP 주소를 공개하지 않음
c.Addresses.NoAnnounce = appendSingle(c.Addresses.NoAnnounce, defaultServerFilte
// 사설 IP 주소로부터의 연결 차단
c.Swarm.AddrFilters = appendSingle(c.Swarm.AddrFilters, defaultServerFilters)
// mDNS 발견 비활성화
c.Discovery.MDNS.Enabled = false
// NAT 포트 매핑 비활성화
c.Swarm.DisableNatPortMap = true
return  nil
},
}
```

필터링되는 주소 범위:

```
var  defaultServerFilters = []string{
"/ip4/10.0.0.0/ipcidr/8", // 사설 네트워크 A 클래스
"/ip4/172.16.0.0/ipcidr/12", // 사설 네트워크 B 클래스
"/ip4/192.168.0.0/ipcidr/16", // 사설 네트워크 C 클래스
"/ip4/169.254.0.0/ipcidr/16", // 링크 로컬
"/ip6/fe80::/ipcidr/10", // IPv6 링크 로컬
// ... 기타 특수 주소 범위들
}
```

local-discovery 프로파일

목적: server 프로파일의 역효과를 되돌리고 로컬 네트워크에서의 발견을 활성화

주요 변경사항:

- 사설 IP 주소 필터 제거
- mDNS 발견 활성화
- NAT 포트 매핑 활성화

```
"local-discovery": {
Description: `Sets default values to fields affected by the server
profile, enables discovery in local networks.`,
Transform: func(c *Config) error {
c.Addresses.NoAnnounce = deleteEntries(c.Addresses.NoAnnounce, defaultServerFilt
c.Swarm.AddrFilters = deleteEntries(c.Swarm.AddrFilters, defaultServerFilters)
c.Discovery.MDNS.Enabled = true
```

```
c.Swarm.DisableNatPortMap = false

return  nil

},

}
```

test 프로파일

목적: 테스트 환경에서 외부 간섭을 최소화

주요 변경사항:

- 모든 주소를 localhost로 제한
- 랜덤 포트 사용 (포트 0)
- 부트스트랩 피어 제거
- mDNS 발견 비활성화

```
"test": {

Description: `Reduces external interference of IPFS daemon, this

is useful when using the daemon in test environments.`,

Transform: func(c *Config) error {

c.Addresses.API = Strings{"/ip4/127.0.0.1/tcp/0"}

c.Addresses.Gateway = Strings{"/ip4/127.0.0.1/tcp/0"}

c.Addresses.Swarm = []string{"/ip4/127.0.0.1/tcp/0"}



c.Swarm.DisableNatPortMap = true

c.Routing.LoopbackAddressesOnLanDHT = True
```

```
c.Bootstrap = []string{}
c.Discovery.MDNS.Enabled = false
c.AutoTLS.Enabled = False
return nil
},
}
```

randomports 프로파일

목적: Swarm에 랜덤 포트 사용

```
"randomports": {
Description: `Use a random port number for swarm.`,
Transform: func(c *Config) error {
port, err := getAvailablePort()
if err != nil {
return err
}
c.Addresses.Swarm = []string{
fmt.Sprintf("/ip4/0.0.0.0/tcp/%d", port),
fmt.Sprintf("/ip6/::/tcp/%d", port),
}
return nil
```

```
},

}
```

데이터스토어 프로파일

flatfs 프로파일 (기본값)

목적: 가장 검증된 안정적인 데이터스토어 사용

특징:

- 각 블록을 개별 파일로 저장
- 파일시스템 신뢰성에 의존
- 가비지 컬렉션 시 즉시 공간 회수
- 메모리 사용량 최소화

```
"flatfs": {

Description: `Configures the node to use the flatfs datastore.



This is the most battle-tested and reliable datastore.

You should use this datastore if:



* You need a very simple and very reliable datastore, and you trust your

filesystem. This datastore stores each block as a separate file in the

underlying filesystem so it's unlikely to loose data unless there's an issue

with the underlying file system.
```

```
* You need to run garbage collection in a way that reclaims free space as soon a
* You want to minimize memory usage.
* You are ok with the default speed of data import, or prefer to use --nocopy.`,

InitOnly: true,
Transform: func(c *Config) error {
c.Datastore.Spec = flatfsSpec()
return  nil
},
}
```

FlatFS 사양:

```
func  flatfsSpec() map[string]interface{} {
return  map[string]interface{}{
"type": "mount",
"mounts": []interface{}{
map[string]interface{}{
"mountpoint": "/blocks",
"type": "flatfs",
"prefix": "flatfs.datastore",
"path": "blocks",
"sync": false,
```

```
"shardFunc": "/repo/flatfs/shard/v1/next-to-last/2",

},

map[string]interface{}{

"mountpoint": "/",

"type": "levelds",

"prefix": "leveldb.datastore",

"path": "datastore",

"compression": "none",

},

},

}

}
```

pebbleds 프로파일

목적: 고성능 데이터스토어 사용

특징:

- LevelDB/RocksDB에서 영감을 받은 고성능 키-값 저장소
- CockroachDB에서 내부적으로 사용
- 멀티 테라바이트 데이터셋에 적합
- 충분한 메모리가 있을 때 성능 향상

```
"pebbleds": {

Description: `Configures the node to use the pebble high-performance datastore.
```

```
Pebble is a LevelDB/RocksDB inspired key-value store focused on performance

and internal usage by CockroachDB.

You should use this datastore if:



- You need a datastore that is focused on performance.

- You need reliability by default, but may choose to disable WAL for maximum per

- This datastore is good for multi-terabyte data sets.

- May benefit from tuning depending on read/write patterns and throughput.

- Performance is helped significantly by running on a system with plenty of memo
```

```
InitOnly: true,

Transform: func(c *Config) error {

c.Datastore.Spec = pebbleSpec()

return  nil

},

}
```

badgerds 프로파일 (레거시)

목적: 레거시 BadgerDB v1 사용 (권장하지 않음)

주의사항:

- 알려진 버그가 있는 구버전

- 업스트림 팀에서 더 이상 지원하지 않음
- 기존 사용자의 마이그레이션을 위해서만 제공

```
"badgerds": {
Description: `Configures the node to use the legacy badgerv1 datastore.

NOTE: this is badger 1.x, which has known bugs and is no longer supported by the
It is provided here only for pre-existing users, allowing them to migrate away t

InitOnly: true,
Transform: func(c *Config) error {
c.Datastore.Spec = badgerSpec()
return  nil
},
}
```

성능 최적화 프로파일

lowpower 프로파일

목적: 시스템 오버헤드 감소 (성능 저하 가능성 있음)

주요 변경사항:

- DHT 서버 모드 비활성화 (클라이언트 모드만)
- AutoNAT 서비스 비활성화

- 릴레이 서비스 비활성화
- 최소한의 연결 유지

```
"lowpower": {
Description: `Reduces daemon overhead on the system. May affect node
functionality - performance of content discovery and data
fetching may be degraded.`,
Transform: func(c *Config) error {
// "서버" 서비스들 비활성화 (dht, autonat, limited relay)
c.Routing.Type = NewOptionalString("autoclient")
c.AutoNAT.ServiceMode = AutoNATServiceDisabled
c.Swarm.RelayService.Enabled = False

// 최소한의 연결만 유지
lowWater := int64(20)
highWater := int64(40)
gracePeriod := time.Minute
c.Swarm.ConnMgr.Type = NewOptionalString("basic")
c.Swarm.ConnMgr.LowWater = &OptionalInteger{value: &lowWater}
c.Swarm.ConnMgr.HighWater = &OptionalInteger{value: &highWater}
c.Swarm.ConnMgr.GracePeriod = &OptionalDuration{&gracePeriod}
return  nil
},
```

```
}
```

DHT 공지 프로파일

announce-off 프로파일

목적: DHT 공지 시스템 비활성화

주의사항:

- 수동 피어링 설정과 함께 사용
- 이 노드만 데이터를 호스팅하는 경우 DHT 기반 검색 불가능

```
"announce-off": {
Description: `Disables Provide and Reprovide systems (announcing to Amino DHT).

USE WITH CAUTION:
The main use case for this is setups with manual Peering.Peers config.
Data from this node will not be announced on the DHT. This will make
DHT-based routing and data retrieval impossible if this node is the only
one hosting it, and other peers are not already connected to it.`,
Transform: func(c *Config) error {
c.Provider.Enabled = False
c.Reprovider.Interval = NewOptionalDuration(0) // 0은 주기적 재공지 비활성화
return  nil
},
```

```
}
```

announce-on 프로파일

목적: announce-off 프로파일 효과 되돌리기

```
"announce-on": {
Description: `Re-enables Provide and Reprovide systems (reverts announce-off pro
Transform: func(c *Config) error {
c.Provider.Enabled = True
c.Reprovider.Interval = NewOptionalDuration(DefaultReproviderInterval)
return  nil
},
}
```

콘텐츠 임포트 프로파일

legacy-cid-v0 프로파일

목적: 레거시 CIDv0 형식 사용

설정:

- CID 버전 0
- Raw leaves 비활성화
- SHA2-256 해시
- 256 KiB 청크 크기

```
"legacy-cid-v0": {
Description: `Makes UnixFS import produce legacy CIDv0 with no raw leaves, sha2-
Transform: func(c *Config) error {
c.Import.CidVersion = *NewOptionalInteger(0)
c.Import.UnixFSRawLeaves = False
c.Import.UnixFSChunker = *NewOptionalString("size-262144")
c.Import.HashFunction = *NewOptionalString("sha2-256")
c.Import.UnixFSFileMaxLinks = *NewOptionalInteger(174)
c.Import.UnixFSDirectoryMaxLinks = *NewOptionalInteger(0)
c.Import.UnixFSHAMTDirectoryMaxFanout = *NewOptionalInteger(256)
c.Import.UnixFSHAMTDirectorySizeThreshold = *NewOptionalString("256KiB")
return  nil
},
}
```

test-cid-v1 프로파일

목적: 현대적인 CIDv1 형식 사용

설정:

- CID 버전 1
- Raw leaves 활성화
- SHA2-256 해시
- 1 MiB 청크 크기

```
"test-cid-v1": {
Description: `Makes UnixFS import produce CIDv1 with raw leaves, sha2-256 and 1
Transform: func(c *Config) error {
c.Import.CidVersion = *NewOptionalInteger(1)
c.Import.UnixFSRawLeaves = True
c.Import.UnixFSChunker = *NewOptionalString("size-1048576")
c.Import.HashFunction = *NewOptionalString("sha2-256")
c.Import.UnixFSFileMaxLinks = *NewOptionalInteger(174)
c.Import.UnixFSDirectoryMaxLinks = *NewOptionalInteger(0)
c.Import.UnixFSHAMTDirectoryMaxFanout = *NewOptionalInteger(256)
c.Import.UnixFSHAMTDirectorySizeThreshold = *NewOptionalString("256KiB")
return  nil
},
}
```

test-cid-v1-wide 프로파일

목적: 더 넓은 DAG 구조를 가진 CIDv1 사용

설정:

- 노드당 최대 1024개 링크
- 1MiB 이상에서 HAMT 디렉토리로 전환

```
"test-cid-v1-wide": {
```

```
Description: `Makes UnixFS import produce CIDv1 with raw leaves, sha2-256 and 1M

Transform: func(c *Config) error {

c.Import.CidVersion = *NewOptionalInteger(1)

c.Import.UnixFSRawLeaves = True

c.Import.UnixFSChunker = *NewOptionalString("size-1048576") // 1MiB

c.Import.HashFunction = *NewOptionalString("sha2-256")

c.Import.UnixFSFileMaxLinks = *NewOptionalInteger(1024)

c.Import.UnixFSDirectoryMaxLinks = *NewOptionalInteger(0)

c.Import.UnixFSHAMTDirectoryMaxFanout = *NewOptionalInteger(1024)

c.Import.UnixFSHAMTDirectorySizeThreshold = *NewOptionalString("1MiB")

return  nil

},

}
```

메트릭 추적 프로파일

*-measure 프로파일들

목적: 데이터스토어 성능 메트릭 수집

제공되는 프로파일:

- `flatfs-measure`
- `pebbleds-measure`
- `badgerds-measure`

기능:

- Prometheus 메트릭 엔드포인트에서 `*_datastore_*` 메트릭 노출
- 성능 모니터링 및 튜닝에 유용

프로파일 사용 가이드

초기화 시 프로파일 적용

```
# 단일 프로파일 적용
ipfs init --profile server

# 여러 프로파일 적용 (쉼표로 구분)
ipfs init --profile server,flatfs-measure

# 고성능 서버 설정
ipfs init --profile server,pebbleds
```

런타임 프로파일 적용

```
# 기존 노드에 프로파일 적용 (InitOnly가 아닌 프로파일만)
ipfs config profile apply lowpower

# 프로파일 효과 되돌리기
ipfs config profile apply announce-on
```

프로파일 조합 권장사항

개발/테스트 환경

```
ipfs init --profile test,flatfs
```

개인 사용자 (홈 네트워크)

```
ipfs init --profile local-discovery,flatfs
```

공용 서버 (클라우드)

```
ipfs init --profile server,pebbleds-measure
```

저전력 디바이스 (IoT, 라즈베리파이)

```
ipfs init --profile lowpower,flatfs
ipfs config profile apply announce-off # 필요시
```

고성능 서버 (대용량 데이터)

```
ipfs init --profile server,pebbleds-measure,test-cid-v1-wide
```

커스텀 프로파일 생성

프로파일 구조 정의

```
// 커스텀 프로파일 예시
config.Profiles["my-custom"] = config.Profile{
Description: "My custom profile for specific use case",
Transform: func(c *config.Config) error {
// 설정 변경 로직
c.Gateway.NoFetch = true
c.Swarm.ConnMgr.LowWater = &config.OptionalInteger{value: &[]int64{50}[0]}
return  nil
},
}
```

프로파일 등록 및 사용

1. **코드 수정**: `config/profile.go` 의 `Profiles` 맵에 추가
2. **빌드**: 수정된 코드로 IPFS 재빌드
3. **사용**: `ipfs init --profile my-custom`

프로파일 충돌 및 우선순위

적용 순서

프로파일은 명령행에서 지정된 순서대로 적용됩니다:

```
# server 프로파일 먼저 적용, 그 다음 lowpower 적용
```

```
ipfs init --profile server,lowpower
```

충돌 해결

- 나중에 적용된 프로파일이 이전 설정을 덮어씀
- 상호 보완적인 프로파일 조합 권장
- 상충되는 프로파일 조합 시 예상치 못한 동작 가능

프로파일 검증

```
# 적용된 설정 확인
ipfs config show

# 특정 설정 값 확인
ipfs config Addresses.Swarm
ipfs config Datastore.Spec
```

이 프로파일 시스템을 통해 사용자는 복잡한 IPFS 설정을 쉽게 관리하고, 다양한 환경과 요구사항에 맞게 노드를 최적화할 수 있습니다.